Assignment 2: HTTP APIs, gRPC and Metrics - Report

Enterprise Software Development, Fall 2026. Individual submission. Repository: https://github.com/romaisashazad/ESD-Assignment-01 (branch `a2-work`, extends Assignment 1 in the same repo).

This report covers the QR code service from Assignment 1 (Flask, Docker Compose, Prometheus, Grafana, ELK). In Assignment 2 I (A) improved the HTTP API, (B) added a gRPC service with the same business rules and one source of truth, and (C) audited and extended the metrics. Screenshot times are local time (UTC+5); the app logs UTC. The commands to run and test everything are in the README.

What changed, in one table

Part	What I built	Where in the repo
A HTTP API	Resource-style API (`POST /v1/qr-codes`, `GET /v1/qr-codes/{id}`), consistent error format, OpenAPI spec used in tests, Idempotency-Key, rate limit with 429 + Retry-After, resilient Python client	services/qr-service/app.py, openapi/openapi.yaml, clients/qr_client.py, demo_reliability.py
B gRPC	New container `qr-grpc-service`, .proto, generated code, handler, Python client with deadline, request-ID in metadata, Filebeat covers both services	services/proto/qr.proto, services/grpc-service/, clients/grpc_client.py
C Metrics	Metrics audit, gRPC metrics scraped by Prometheus, bounded labels, new Grafana panels, client-vs-server duration discussion	telemetry/prometheus, telemetry/grafana/dashboards/qr-service.json

Part A: HTTP API improvement

A1. REST constraints (six constraints)

Constraint	Status in my API	Evidence / gap
Client-server	Met. Client and service are separate; the service owns storage and rendering.	Postman, curl and the Python client all use the same API.
Stateless	Met. Every request carries what it needs (body, headers). No sessions. The rate limiter keeps counters in memory but that is not client state.	Gap: counters are per process, so with several replicas each would count separately.
Cacheable	Met for GET: `Cache-Control: public, max-age=3600` on a stored PNG (an id never changes). POST is not cacheable. Legacy `/generate` sends `no-cache`.	Screenshot of GET headers below.
Uniform interface	Mostly met. Nouns in the path (`/v1/qr-codes`), the HTTP verb gives the action, standard codes (201, 400, 404, 429), a Location header and one JSON error shape.	Gap: only POST and GET exist; no PUT/DELETE because QR codes are immutable here.
Layered system	Met. A client cannot tell whether it talks to the service directly or via a proxy; it only sees the public URL.	No gateway is used in this project.
Code on demand (optional)	Not used. The service never sends code to the client.	Optional constraint, so no gap.

A2. Endpoint review

Endpoint	Problem found	Fix (Assignment 2)
`POST /generate`	Verb in the path; returns raw PNG with 200; nothing to point to afterwards; inconsistent errors.	Kept for old callers but marked deprecated (see before/after). New resource endpoint added.

Endpoint	Problem found	Fix (Assignment 2)
`POST /v1/qr-codes` (new)	-	Creates a QR code, stores it, returns **201** + `Location` + JSON (`id, kind, text_length, created_at, image_url`).
`GET /v1/qr-codes/{id}` (new)	-	Returns the PNG (200) or a JSON 404. Cacheable.
`/health`, `/metrics`, `/admin/fault`, `/demo`	Operational endpoints, not part of the public resource API. /health and /metrics are exempt from rate limiting so monitoring is never blocked.	Documented separately; /health is in the OpenAPI file.

Status codes used: 200 (GET ok), 201 (created, also for an idempotent replay), 400 (bad input), 404 (unknown id), 405 (wrong method), 413 (body too large), 415 (wrong content type), 422 (idempotency key reused with different text), 429 (rate limited). **Error format** is the same everywhere: `{"error": {"code", "message", "request_id"}}`, so a client can switch on `code` and quote `request_id` when asking for help (it matches the log line).

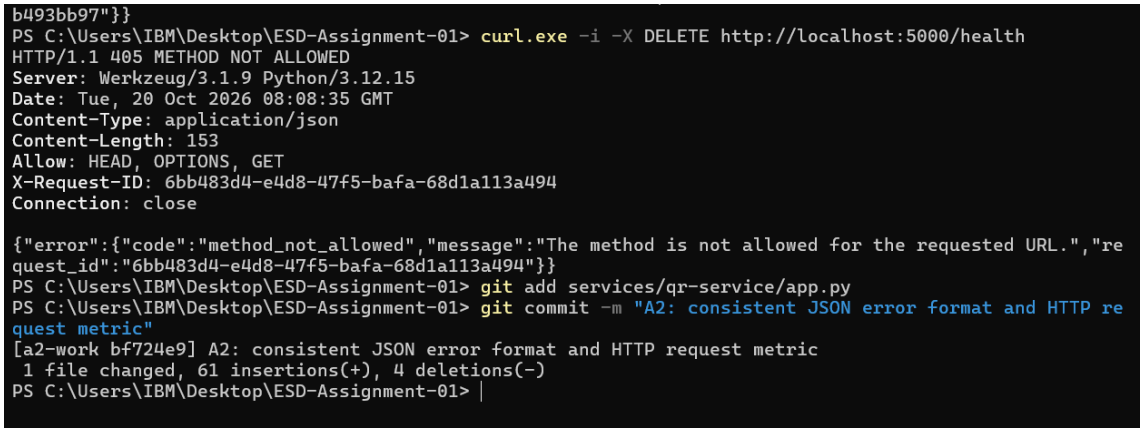


Figure A1. A wrong method (DELETE /health) returns 405 in the same JSON error format, with an Allow header and a request_id.



Figure A2. Valid POST /v1/qr-codes returns 201 Created with a Location header and the JSON resource. The second request below repeats the same Idempotency-Key and gets the same id plus Idempotent-Replayed: true (A4).

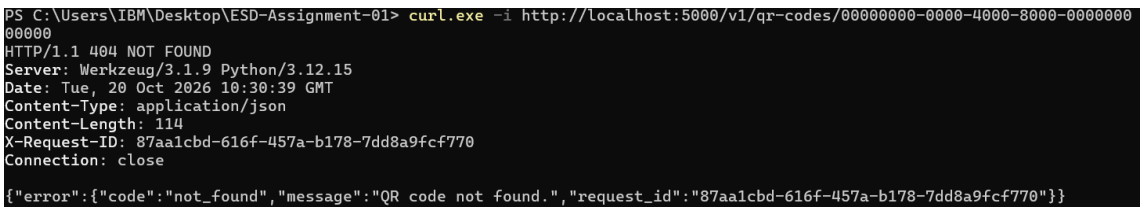


Figure A3. GET of an unknown id returns 404 with the standard error body.

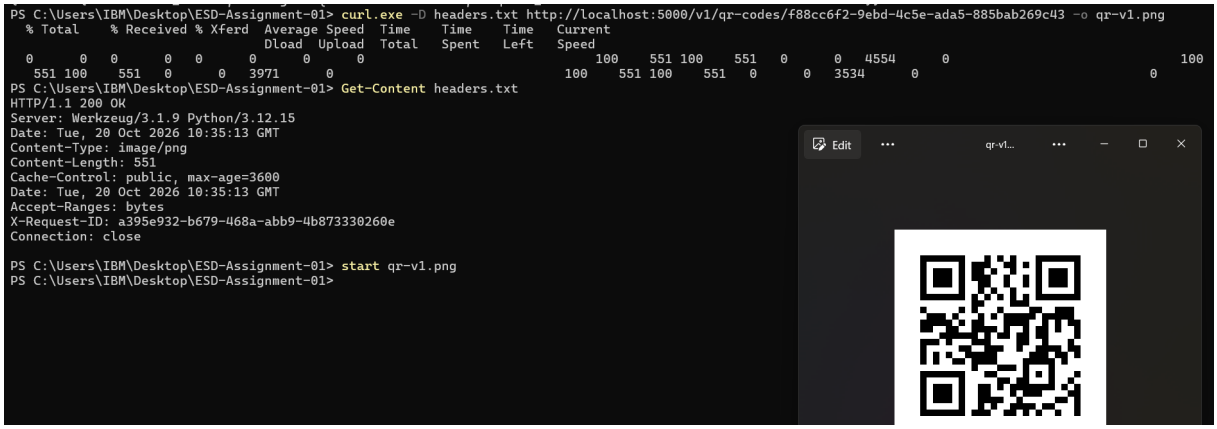


Figure A4. GET of a stored id returns 200 image/png with Cache-Control: public, max-age=3600 (cacheable). The QR image opened on the right.

A3. OpenAPI spec, and how I used it

What it is (purpose): the OpenAPI Specification is a standard, machine-readable file (YAML or JSON) that describes an HTTP API: its paths, methods, inputs, and success and error responses. People and tools use it as a shared contract: to generate documentation, to import requests into tools like Postman, to generate client code, and to test that a server really behaves as described.

`openapi/openapi.yaml` (OpenAPI 3.0.3) describes every public endpoint: parameters (`Idempotency-Key`, `X-Request-ID`), request body, all response codes, response headers (Location, Retry-After, Idempotent-Replayed) and the Error, QrCode and CreateRequest schemas. It is the written contract between the service and its callers.

How it was used to test (not just written): (1) I imported it into Postman, which generated the requests and a collection from the spec (Figures A5-A7). (2) The automated test `test_responses_match_openapi_schemas` loads the YAML, calls the real API, and validates the live responses against the schemas with `jsonschema`. If the code and the spec drift apart, the test fails.

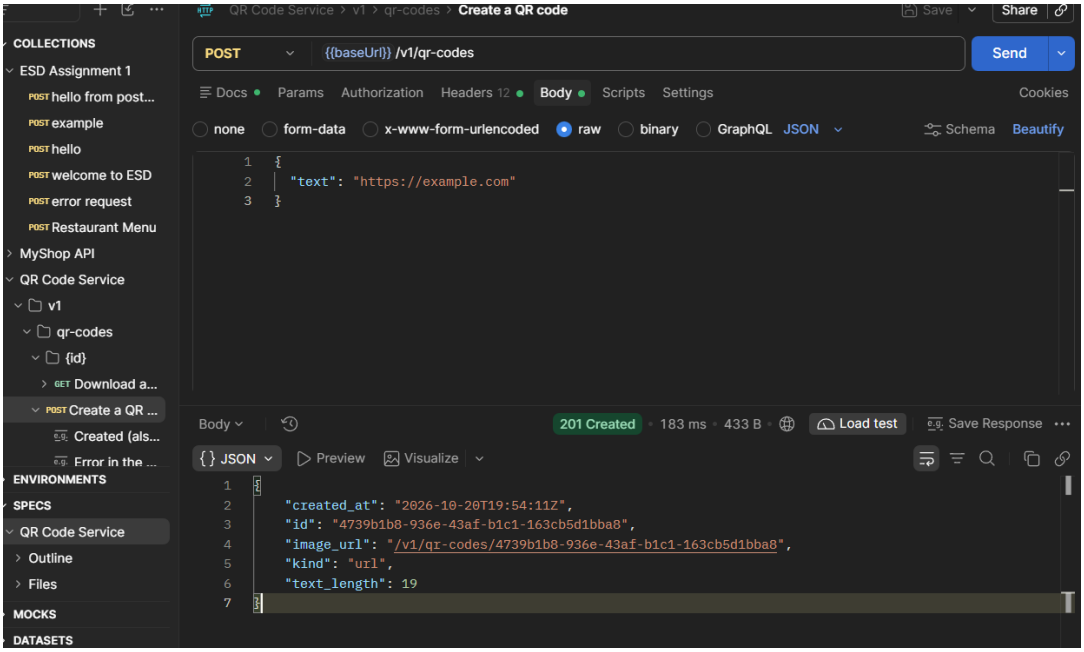


Figure A5. Postman, imported from the OpenAPI file: Create a QR code returns 201 Created with the JSON body.

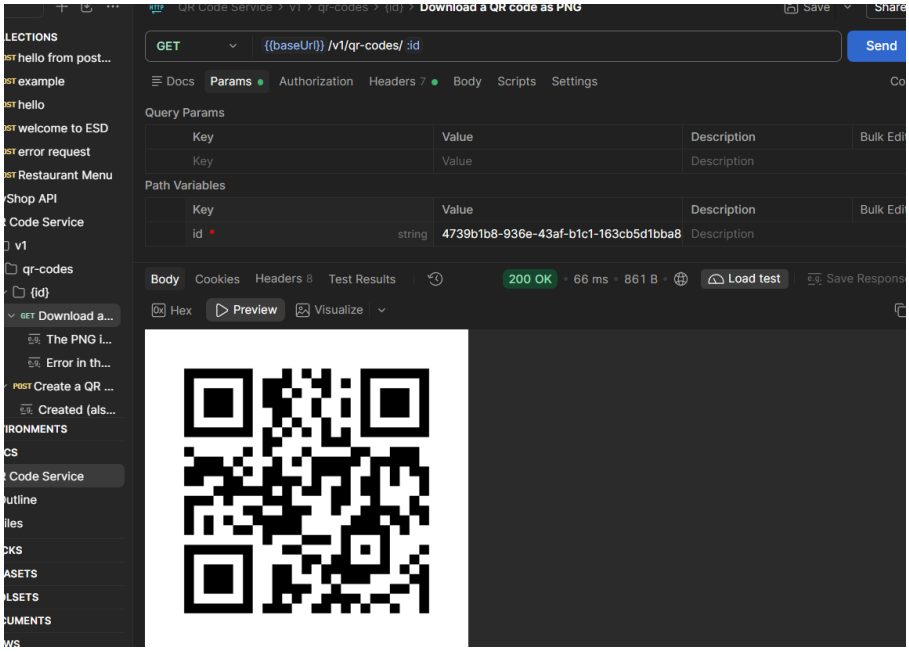


Figure A6. Postman: "Download a QR code as PNG" with the id path variable returns 200 and the image preview.

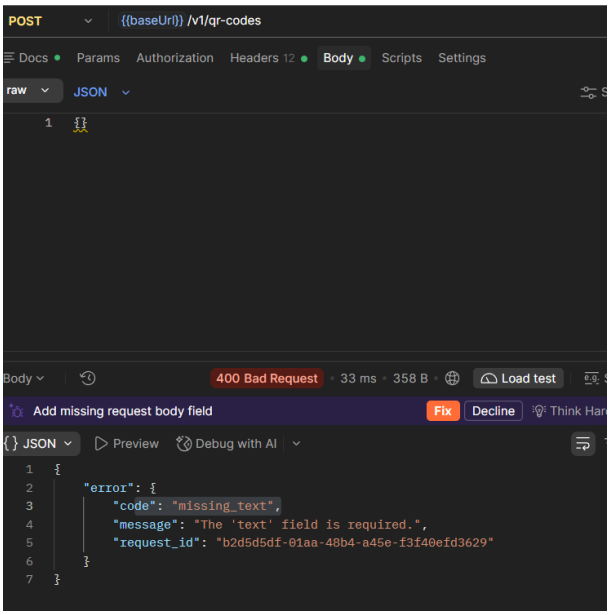


Figure A7. Postman: an empty body returns 400 with error code missing_text and a request_id.

A4. Idempotency

Problem: POST is not safe to retry. If a client times out and sends the same request again, it could create two QR codes. **Solution:** the client may send an `Idempotency-Key` header. The first request with a key creates and stores the QR code. A later request with the same key and the same text returns the *same* resource (same id, still 201) with `Idempotent-Replayed: true`, and nothing new is created or counted. The same key with different text returns **422 idempotency_key_reused**, because that is a client mistake. Without a key every request creates a new resource. The key and its result are stored in SQLite inside a transaction (`BEGIN IMMEDIATE`, then re-check), so two simultaneous requests with the same key cannot both create a code. Proof: Figure A2 (same id on replay), and tests `test_idempotency_replay_same_resource_not_double_counted`, `test_key_reuse_with_different_text_422`, `test_without_key_repeat_creates_new_resource`.

A5. Rate limiting

Rule: at most **30 requests per 10 seconds per client IP** (sliding window, in memory). When the limit is exceeded the service answers **429 Too Many Requests** with a `Retry-After` header (seconds until a request will be accepted again) and the standard JSON error (`rate_limited`); `qr_rate_limited_total` counts rejections. **Notes:** the limit is shared by everyone behind one IP, and from Docker all requests from my host look like one IP. It lives in one process, so it is a demo limit, not a distributed one. /health and /metrics are exempt.

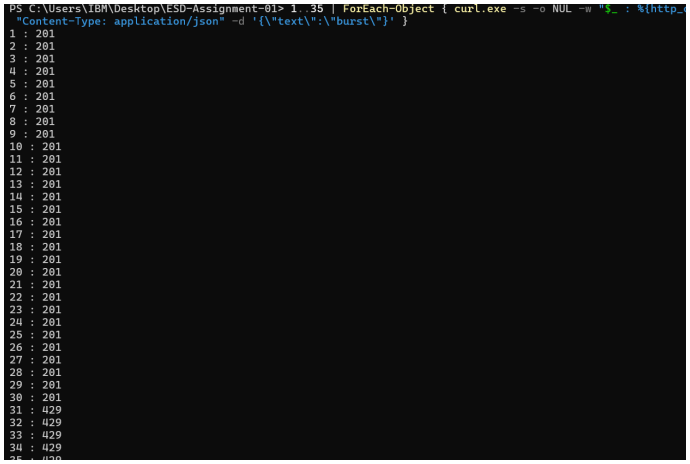


Figure A8. Burst of 35 requests: the first 30 get 201, the rest get 429.

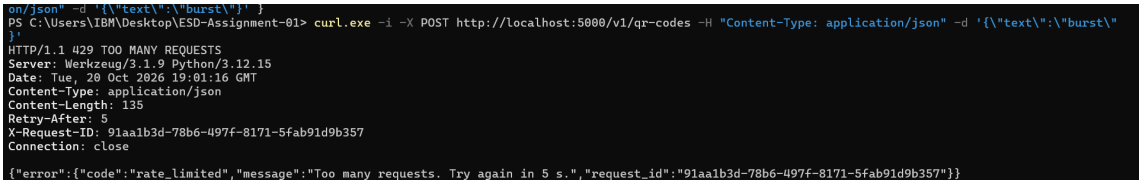


Figure A9. A rejected request: 429, Retry-After header, error code rate_limited.



Figure A10. After waiting out the window the same call is accepted (201) again.

A6. Before / after

User task chosen: "turn some text into a QR code and get the image".

Old endpoint	Problem	Change	Reason
`POST /generate` (returns a PNG, nothing stored)	Verb in the URL; 200 with raw image and no handle to refer to it later; a retry creates a duplicate; no limit on call volume; errors not consistent	`POST /v1/qr-codes` returns 201 + Location + JSON; `GET /v1/qr-codes/{id}` returns the PNG; Idempotency-Key; rate limit; one error format	Resource-style design is predictable, safe to retry and protects the service; the old path stays (deprecated) so existing callers do not break

Other endpoints: /health and /metrics unchanged (also exempt from rate limiting); /admin/fault and /demo unchanged (internal, used for the fault and cardinality demos); `/generate` kept but deprecated. **Callers updated:** the load-test script now targets `/v1/qr-codes`, and the new Python client and Postman collection use the new endpoints. Tests cover a valid request, invalid input and a repeated request (Section A7).

Aspect	Before (A1: POST /generate)	After (A2)
URL design	Verb in path: `/generate`	Resource: `POST /v1/qr-codes`, `GET /v1/qr-codes/{id}`, versioned (`/v1`)
Result of create	200 + raw PNG, nothing stored	201 + Location + JSON resource; image fetched later by id

Aspect	Before (A1: POST /generate)	After (A2)
Retries	A retry makes a duplicate	Idempotency-Key: replay returns the same resource
Errors	Inconsistent	One format with code, message, request_id; correct status codes (400/404/405/413/415/422/429)
Overload	No protection	Rate limit, 429 + Retry-After
Caching	None	GET has Cache-Control
Contract	Only README text	OpenAPI file, checked by a test
Old clients	-	`/generate` still works but sends `Deprecation: true` and `Link: </v1/qr-codes>; rel="successor-version"`

Deprecated means "still works, but do not build new things on it; it will be removed later". The headers tell callers where to go (successor-version) so they can migrate without breaking.

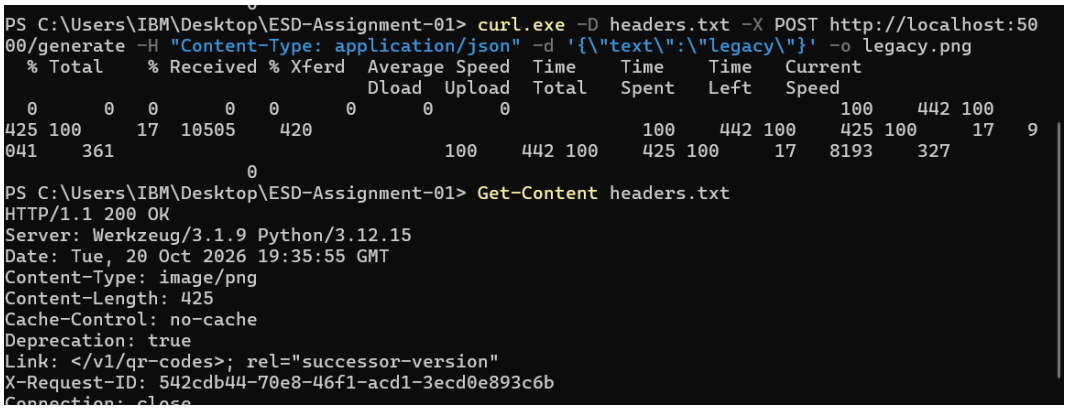


Figure A11. Legacy POST /generate still returns 200 but now carries Deprecation: true and a Link to the successor.

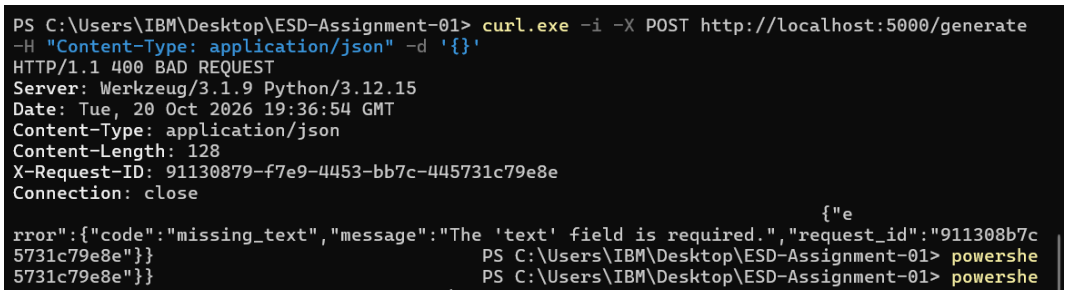


Figure A12. Legacy endpoint errors now use the new JSON error format.

Load check (regression): a one-minute load test (one request every 0.5 s) against the new endpoint: 111 requests, 0 failures, average 37 ms and p95 50 ms measured at the client (includes network), none slower than 400 ms.

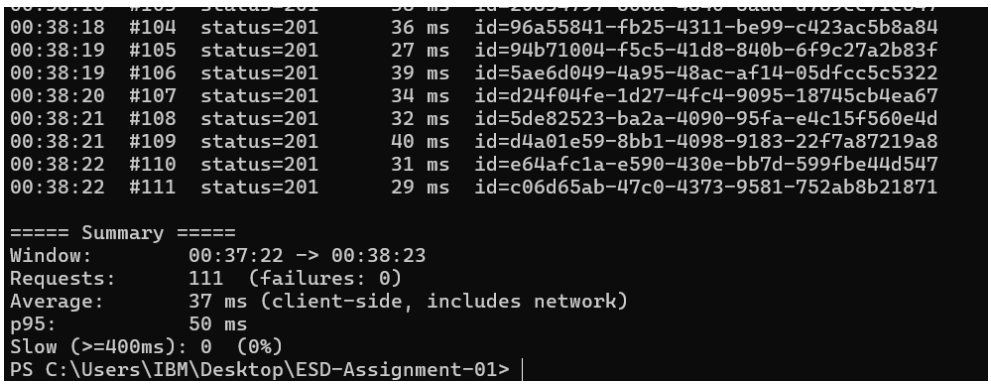


Figure A13. Load test summary (end of the run).

A7. Tests

`services/qr-service/tests/test_api.py` has 19 tests: valid create, 5 invalid bodies, wrong content type (415), body too large (413), unknown/bad id (404), wrong method (405 with Allow), idempotency (3 tests), bad key (400), rate limit 429 + Retry-After then recovery, per-IP limit, health/metrics not limited, legacy endpoint still works and is deprecated, and responses match the OpenAPI schemas.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker run --rm -e PYTHONDONTWRITEBYTECODE=1 -v "${PWD}/services:/app/services"
v "${PWD}/openapi:/app/openapi" -w /app/services/qr-service python:3.12-slim sh -c "pip install -q --root-user-action=ignor
 --disable-pip-version-check -r requirements.txt pytest pyyaml jsonschema && python -m pytest tests -v -p no:cacheprovider"
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.7.0 -- /usr/local/bin/python
rootdir: /app/services/qr-service
collecting ... collected 19 items

tests/test_api.py::test_create_valid_returns_201_and_location PASSED     [  5%]
tests/test_api.py::test_invalid_bodies[body0-400-missing_text] PASSED    [ 10%]
tests/test_api.py::test_invalid_bodies[body1-400-invalid_text] PASSED    [ 15%]
tests/test_api.py::test_invalid_bodies[body2-400-invalid_text] PASSED    [ 21%]
tests/test_api.py::test_invalid_bodies[body3-400-text_too_long] PASSED   [ 26%]
tests/test_api.py::test_invalid_bodies[body4-400-invalid_json] PASSED    [ 31%]
tests/test_api.py::test_wrong_content_type_415 PASSED                    [ 36%]
tests/test_api.py::test_body_too_large_413 PASSED                        [ 42%]
tests/test_api.py::test_unknown_id_404_and_bad_id_404 PASSED             [ 47%]
tests/test_api.py::test_wrong_method_405_has_allow PASSED                [ 52%]
tests/test_api.py::test_idempotency_replay_same_resource_not_double_counted PASSED [ 57%]
tests/test_api.py::test_without_key_repeat_creates_new_resource PASSED   [ 63%]
tests/test_api.py::test_key_reuse_with_different_text_422 PASSED         [ 68%]
tests/test_api.py::test_bad_key_400 PASSED                               [ 73%]
tests/test_api.py::test_rate_limit_429_with_retry_after_then_recovers PASSED [ 78%]
tests/test_api.py::test_rate_limit_is_per_ip PASSED                      [ 84%]
tests/test_api.py::test_health_and_metrics_not_rate_limited PASSED       [ 89%]
tests/test_api.py::test_legacy_generate_still_works_and_is_deprecated PASSED [ 94%]
tests/test_api.py::test_responses_match_openapi_schemas PASSED           [100%]

============================== 19 passed in 3.08s ==============================
PS C:\Users\IBM\Desktop\ESD-Assignment-01>
```

Figure A14. All 19 HTTP tests pass (run in a container with the command from the README).

A8. Python caller: timeout, retry, backoff, circuit breaker

`clients/qr_client.py` is a small class around `requests`. Settings and the reason for each:

Setting	Value	Why
Timeout	1 s per attempt	Never wait forever for a slow server.
Retries	3 attempts in total	Handles short glitches.
Backoff	0.2 s, then 0.4 s, plus up to 50% random jitter	Waiting longer each time gives the server room; jitter stops many clients retrying at the same moment.
Idempotency-Key on retries	Same key on every attempt	A retry after a timeout cannot create a duplicate (see A4).
Circuit breaker	Opens after 3 consecutive failures; stays open 10 s; then half-open (one trial call)	While open, calls fail immediately without hitting the struggling server. A successful trial closes it again.
429 handling	Respects Retry-After; does not count towards the breaker	The server is healthy and just asking us to slow down.

Results (demo script `demo_reliability.py`, using the service fault switch to make the server slow):

- **Success:** healthy server, first attempt succeeds (Figure A15).
- **Retry:** first attempt times out, backoff 0.25 s, second attempt succeeds, and because the key is the same only one resource exists (Figure A16).
- **Breaker:** three timed-out attempts open the breaker (CLOSED to OPEN). The next call fails in 0.000 s without calling the server. After the 10 s open period one trial call goes through (HALF_OPEN) and succeeds, closing the breaker (Figure A17).
- **Rate limit:** a burst of 35 gives 30 accepted and 5 rejected with Retry-After 10 s; the client waits as told and the next call succeeds (Figure A18).

```
NDONTWRITEBYTECODE=1 -v "${PWD}/clients:/clients" -w /clients python:3.12-slim sh -c "pip install -q --root-user-ac
ion=ignore --disable-pip-version-check -r requirements.txt && python -u demo_reliability.py success"
20:12:26 call 1 (healthy server)
20:12:26   attempt 1/3 ok: 201 id=1e66322d-8dcd-4d55-95b6-9517367b8c27 replayed=false
```

Figure A15. Client, healthy server.

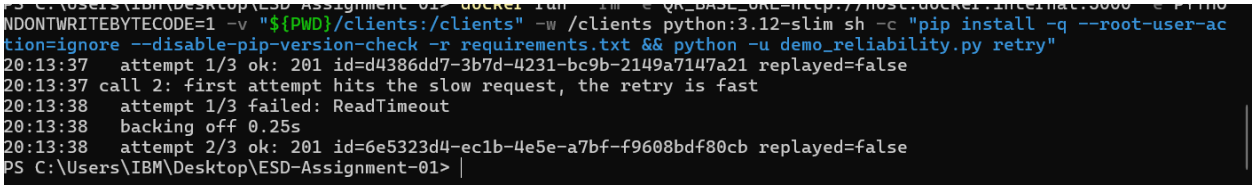


Figure A16. Client retry: first attempt times out, second succeeds.

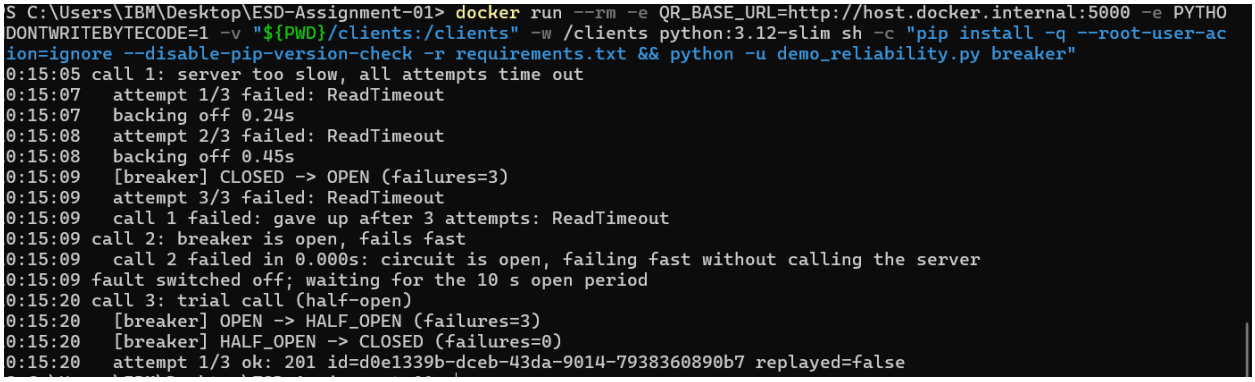


Figure A17. Circuit breaker: CLOSED to OPEN to HALF_OPEN to CLOSED.

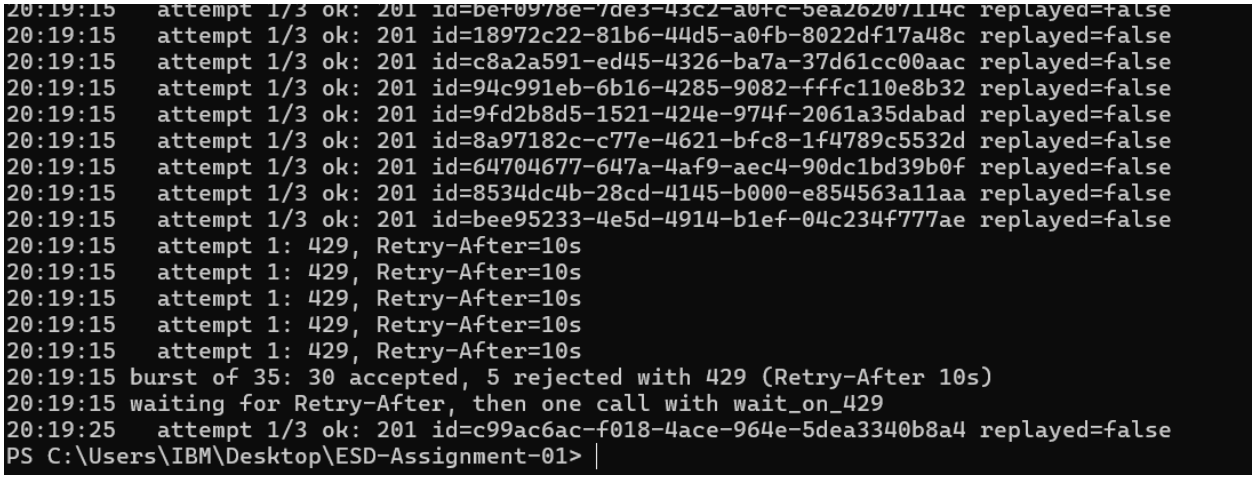


Figure A18. Rate limit seen from the client: 30 accepted, 5 rejected (429, Retry-After 10 s), then a successful call after waiting.

Part B: gRPC service

B1. Design in short

A second container, `qr-grpc-service`, exposes one unary RPC, `CreateQrCode`, defined in `services/proto/qr.proto` (package `qr.v1`). Request: `text`, `idempotency_key`. Response: `id, kind, text_length, created_at, image_url, replayed`. I only wrapped creation (the rubric needs one call that can succeed and fail); fetching the image stays on HTTP.

Same business rules, one source of truth: both services import the same module `common/qrcore.py` (validation, max length 1000, idempotency, rendering) and write to the same SQLite database on a shared Docker volume (`qr-data`). So a code created over gRPC can be fetched over HTTP, and a key used on one side is recognised on the other (replays count in both). The rules are written once, so the two APIs cannot disagree.

Generation command (also in the README), using grpcio-tools 1.66.2:

```
python -m grpc_tools.protoc -I services/proto --python_out=services/grpc-service \
    --grpc_python_out=services/grpc-service services/proto/qr.proto
```

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> dir services\grpc-service


    Directory: C:\Users\IBM\Desktop\ESD-Assignment-01\services\grpc-service


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/21/2026   1:39 AM           1885 qr_pb2.py
-a----        10/21/2026   1:39 AM           3462 qr_pb2_grpc.py


PS C:\Users\IBM\Desktop\ESD-Assignment-01>
```

Figure B1. Generated files qr_pb2.py (messages) and qr_pb2_grpc.py (stub and servicer) in services/grpc-service.

Stub vs handler: the *stub* is generated client code, so the caller just writes `stub.CreateQrCode(request)` like a normal function. The *handler* (servicer method in `server.py`) is the server code that actually runs, calls `qrcore` and returns the response or aborts with a status.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker compose ps
NAME              IMAGE                                              COMMAND                  SERVICE
CREATED           STATUS             PORTS
elasticsearch     docker.elastic.co/elasticsearch/elasticsearch:8.15.0   "/bin/tini -- /usr/l…"   elasticsearch
3 weeks ago       Up About an hour   0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
filebeat          docker.elastic.co/beats/filebeat:8.15.0            "/usr/bin/tini -- /u…"   filebeat
3 weeks ago       Up 5 seconds
grafana           grafana/grafana:latest                             "/run.sh"                grafana
3 weeks ago       Up About an hour   0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
kibana            docker.elastic.co/kibana/kibana:8.15.0             "/bin/tini -- /usr/l…"   kibana
3 weeks ago       Up About an hour   0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
node-exporter     prom/node-exporter:latest                          "/bin/node_exporter"     node-exporter
3 weeks ago       Up About an hour   0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp
prometheus        prom/prometheus:latest                             "/bin/prometheus --c…"   prometheus
3 weeks ago       Up About an hour   0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
qr-grpc-service   esd-assignment-01-qr-grpc-service                  "python -u server.py"    qr-grpc-service
10 seconds ago    Up 9 seconds       0.0.0.0:50051->50051/tcp, [::]:50051->50051/tcp
qr-service        esd-assignment-01-qr-service                       "python app.py"          qr-service
21 minutes ago    Up 21 minutes      0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker compose logs qr-grpc-service --tail 5
qr-grpc-service  | {"@timestamp": "2026-10-20T20:49:22Z", "service.name": "qr-grpc-service", "log.level": "INFO", "m
essage": "gRPC server listening on 50051"}
```

Figure B2. Eight containers are up; qr-grpc-service listens on 50051 (gRPC) and 8000 (metrics). Log line: "gRPC server listening on 50051".

B2. Call-path diagram

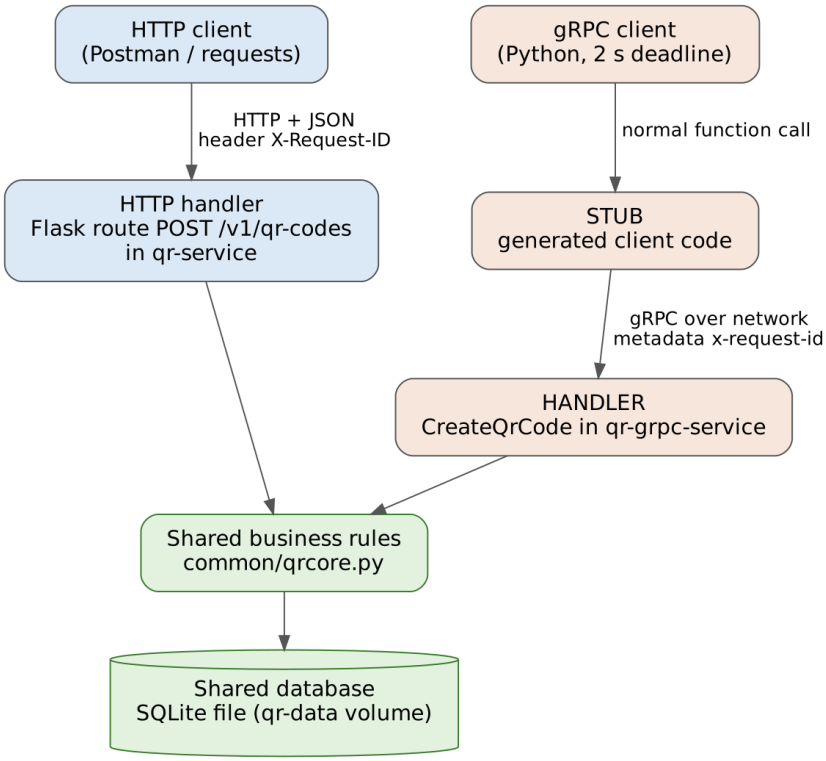


Figure B3. Call paths. Left: HTTP client to Flask handler. Right: gRPC client calls the generated stub, which sends the call over the network to the server handler. Both handlers use the same qrcore rules and the same database.

B3. Same request over HTTP and gRPC: success and failure

`clients/compare_http_grpc.py` sends the same input to both services. gRPC deadline: 2.0 s. The status mapping is chosen so that the same mistake means the same thing in both worlds:

Situation	HTTP	gRPC status	Why this status
Created	201 Created	OK	Success
Empty / missing / too long text, bad key	400 Bad Request (invalid_text, text_too_long, ...)	INVALID_ARGUMENT	Client sent bad input
Idempotency key reused with different text	422 Unprocessable	FAILED_PRECONDITION	Input is well-formed but conflicts with stored state
Unknown id	404 Not Found	NOT_FOUND	Resource does not exist
Server too slow	client timeout (1 s)	DEADLINE_EXCEEDED	Caller's deadline expired
Unexpected bug	500	INTERNAL	Server fault

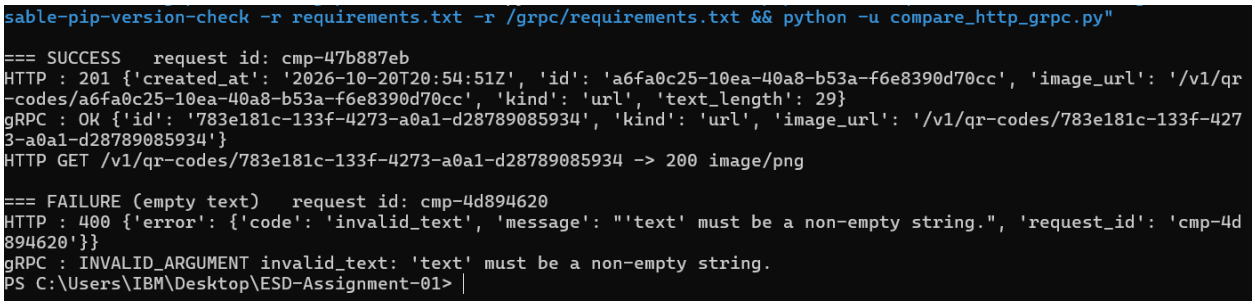


Figure B4. Success: HTTP 201 and gRPC OK (both create a QR code; the HTTP GET then returns the image with 200). Failure with empty text: HTTP 400 invalid_text, gRPC INVALID_ARGUMENT invalid_text. Each call has its own request id (cmp-...).

B4. Request-ID correlation and Filebeat

Each call carries a request id: HTTP uses the `X-Request-ID` header, gRPC uses the metadata key `x-request-id`. The server logs it as `request.id` in JSON; if the caller sends none, the server generates one and returns it. **Filebeat update:** its filter used to keep only `qr-service` logs; it now keeps `qr-service` and `qr-grpc-service` (and the grpc container has the `co.elastic.logs/enabled` label), so both services appear in Kibana. Searching one request id in Kibana returns one log line from each container (the id `cmp-47b887eb` from Figure B4): the HTTP service line and the gRPC service line, so one user action can be followed across both.



Figure B5. Kibana, search request.id:"cmp-47b887eb": 2 documents. The expanded field shows container service qr-service (HTTP side).

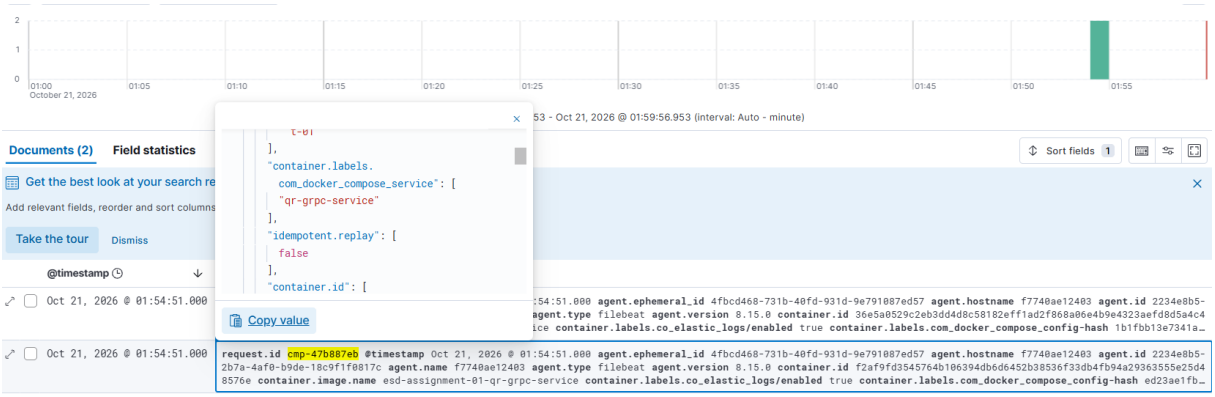


Figure B6. Same search, the second document: container service qr-grpc-service (gRPC side, idempotent.replay false). Same request id in both services.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker run --rm -e PYTHONDONTWRITEBYTECODE=1 -v "${PWD}/services:/app/services" -w /app/services/grpc-service python:3.12-slim sh -c "pip install -q --root-user-action=ignore --disable-pip-version-check -r requirements.txt qrcode==7.4.2 Pillow==10.4.0 pytest && python -m pytest tests -v -p no:cacheprovider"
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.7.0 -- /usr/local/bin/python
rootdir: /app/services/grpc-service
collecting ... collected 6 items

tests/test_grpc.py::test_success_returns_record_and_same_store_as_http PASSED [ 16%]
tests/test_grpc.py::test_invalid_text_is_invalid_argument[empty_text] PASSED [ 33%]
tests/test_grpc.py::test_invalid_text_is_invalid_argument[too_long] PASSED [ 50%]
tests/test_grpc.py::test_idempotency_replay_and_conflict PASSED          [ 66%]
tests/test_grpc.py::test_request_id_round_trips_in_metadata PASSED       [ 83%]
tests/test_grpc.py::test_request_id_generated_when_missing PASSED        [100%]

============================== 6 passed in 0.90s ===============================
```

Figure B7. gRPC tests: 6 passed (success same store as HTTP, invalid text, too-long text, idempotency and conflict, request id round trip in metadata, request id generated when missing).

Part C: Metrics

C1. Audit of the existing metrics (gap table)

Existing metric	Useful?	Gap found	Action
qr_generated_total{kind}	Yes: throughput by kind (url/text)	Counted only creations, not HTTP outcomes (no way to see 4xx/429)	Kept; added http_requests_total
qr_generation_errors_total{reason}	Yes	Series appeared only after the first error, so graphs showed "No data"	Pre-created every reason from the shared ERROR_CODES list (bounded, 7 values)
qr_in_progress (gauge)	Partly	Requests are so fast (tens of ms) that a 5 s scrape almost never sees a value other than 0	Kept, limitation documented below
qr_generation_duration_seconds (histogram)	Partly	Old buckets did not suit a 30-40 ms service: nearly all requests fell in a single bucket, so p95 from histogram_quantile was inaccurate	Re-chose buckets around the real latency range
qr_generation_duration_summary	Average only	A summary cannot be combined across instances	Kept for the average panel
(missing) per-route status counts	-	No way to graph error rate per status, or see 429s	Added http_requests_total{method,route,status}
(missing) gRPC metrics	-	The gRPC service had none	Added grpc_server_* (see C2)
(missing) idempotency / rate limit	-	Replays and rejections invisible	Added qr_idempotent_replays_total and qr_rate_limited_total

The scrape endpoint was checked directly (curl on /metrics) before and after the changes; Prometheus target health is in Figure C2.

C2. Documented new metrics

Metric	Type, unit	Labels	Purpose	Code location	Query
http_requests_total	Counter, responses	method, route (template), status	Throughput and error mix per route	qr-service/app.py, after_request hook	sum by (route, status) (rate(http_requests_total{route!="/health"}[1m]))
qr_idempotent_replays_total	Counter, requests	none	How often retries are absorbed (both services)	app.py (create path) and grpc-service/server.py (_create)	sum by (job) (rate(qr_idempotent_replays_total[5m])) * 60
qr_rate_limited_total	Counter, requests	none	Requests rejected with 429	app.py, rate limiter	rate(qr_rate_limited_total[1m])
grpc_server_calls_total	Counter, calls	method, code (gRPC status)	Calls and error rate by method and status	grpc-service/server.py, CreateQrCode wrapper (finally block)	sum by (method, code) (rate(grpc_server_calls_total[1m]))
grpc_server_call_duration_seconds	Histogram, seconds	method	Latency distribution; p50/p95; heatmap	grpc-service/server.py, CreateQrCode wrapper	histogram_quantile(0.95, sum by (le) (rate(grpc_server_call_duration_seconds_bucket[5m])))
grpc_server_in_flight	Gauge, calls	method	Calls being handled now	grpc-service/server.py, CreateQrCode wrapper	grpc_server_in_flight

Bounded labels: I never use request id, text, QR id, idempotency key, client IP or the raw URL as a label, because each distinct value creates a new time series and memory grows without limit. `route` is the Flask route pattern (e.g. `/v1/qr-codes/<qr_id>`), so all ids share one series. `code` and `reason` come from fixed lists. The series are created at start-up with 0 so panels show 0 instead of "No data".

Bucket choice (gRPC duration histogram): 5 ms, 10, 20, 30, 40, 50, 75, 100, 150, 200, 300, 400, 500, 550, 600, 750 ms, 1 s, 2.5 s. Typical calls take 20-50 ms, so I put fine steps there so that `histogram_quantile` interpolates inside a narrow bucket; wider buckets above catch slow outliers and the fault-injection delay.

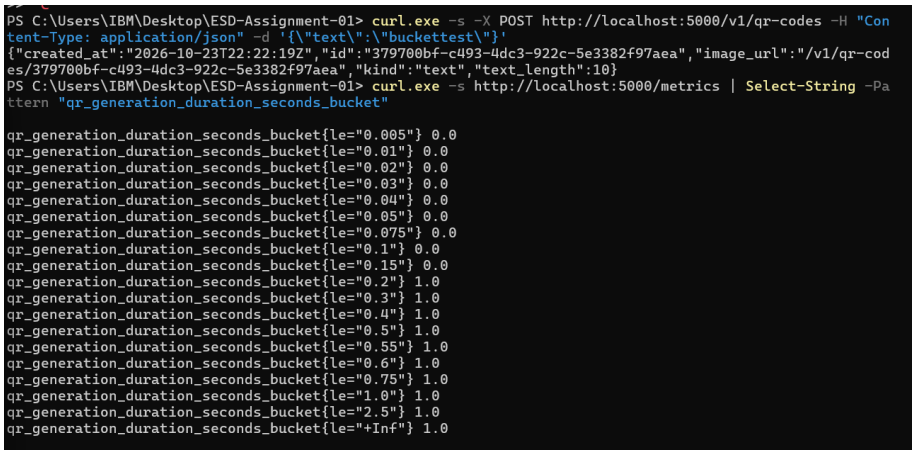


Figure C1. Raw /metrics output for the histogram buckets (le="0.005" up to +Inf) after a single test call. The bucket cumulative counts jump at le="0.2", where that call fell.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker run --rm -e QR_BASE_URL=http://host.docker.internal:50
00 -e GRPC_TARGET=host.docker.internal:50051 -e PYTHONPATH=/grpc -e PYTHONDONTWRITEBYTECODE=1 -v "${PWD}
/clients:/clients" -v "${PWD}/services/grpc-service:/grpc" -w /clients python:3.12-slim sh -c "pip insta
ll -q --root-user-action=ignore --disable-pip-version-check -r requirements.txt -r /grpc/requirements.tx
t && python -u compare_http_grpc.py"

=== SUCCESS   request id: cmp-486c3d29
HTTP : 201 {'created_at': '2026-10-23T22:49:07Z', 'id': '3a8792ea-9ad4-4280-bb80-26d409171945', 'image_u
rl': '/v1/qr-codes/3a8792ea-9ad4-4280-bb80-26d409171945', 'kind': 'url', 'text_length': 29}
gRPC : OK {'id': 'cc4c01dd-ba7e-4966-a2d1-5ccf0b658e0f', 'kind': 'url', 'image_url': '/v1/qr-codes/cc4c0
1dd-ba7e-4966-a2d1-5ccf0b658e0f'}
HTTP GET /v1/qr-codes/cc4c01dd-ba7e-4966-a2d1-5ccf0b658e0f -> 200 image/png

=== FAILURE (empty text)   request id: cmp-358fbd9b
HTTP : 400 {'error': {'code': 'invalid_text', 'message': "'text' must be a non-empty string.", 'request_
id': 'cmp-358fbd9b'}}
gRPC : INVALID_ARGUMENT invalid_text: 'text' must be a non-empty string.
PS C:\Users\IBM\Desktop\ESD-Assignment-01> curl.exe -s http://localhost:8000/metrics | Select-String -Pa
ttern "^(grpc_server|qr_idempotent)" | Select-String -NotMatch "_bucket|_created"

grpc_server_calls_total{code="OK",method="CreateQrCode"} 1.0
grpc_server_calls_total{code="INVALID_ARGUMENT",method="CreateQrCode"} 1.0
grpc_server_calls_total{code="FAILED_PRECONDITION",method="CreateQrCode"} 0.0
grpc_server_calls_total{code="INTERNAL",method="CreateQrCode"} 0.0
grpc_server_call_duration_seconds_count{method="CreateQrCode"} 2.0
grpc_server_call_duration_seconds_sum{method="CreateQrCode"} 0.16614818572998047
grpc_server_in_flight{method="CreateQrCode"} 0.0
qr_idempotent_replays_total 0.0
```

Figure C2a. After HTTP and gRPC calls, scrape output shows grpc_server_calls_total by code (OK 1, INVALID_ARGUMENT 1), duration count/sum and in-flight 0.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> curl.exe -s -X POST http://localhost:5000/v1/qr-co
tent-Type: application/json" -H "Idempotency-Key: replay-1" -d '{\"text\":\"replaytest\"}'
{"created_at":"2026-10-23T22:54:43Z","id":"3efd4e42-85fe-47dc-992a-b0418d246404","image_url":
es/3efd4e42-85fe-47dc-992a-b0418d246404","kind":"text","text_length":10}
PS C:\Users\IBM\Desktop\ESD-Assignment-01> curl.exe -s -X POST http://localhost:5000/v1/qr-co
tent-Type: application/json" -H "Idempotency-Key: replay-1" -d '{\"text\":\"replaytest\"}'
{"created_at":"2026-10-23T22:54:43Z","id":"3efd4e42-85fe-47dc-992a-b0418d246404","image_url":
es/3efd4e42-85fe-47dc-992a-b0418d246404","kind":"text","text_length":10}
PS C:\Users\IBM\Desktop\ESD-Assignment-01> curl.exe -s http://localhost:5000/metrics | Select
ttern "^qr_idempotent_replays_total"

qr_idempotent_replays_total 1.0
```

Figure C2b. Replaying the same Idempotency-Key twice raised qr_idempotent_replays_total to 1.

C3. Prometheus scrapes the gRPC service

`telemetry/prometheus/prometheus.yml` now has three jobs, every 5 s: `qr-service:5000`, `qr-grpc-service:8000` (new) and `node-exporter:9100`. The gRPC process serves its metrics on a separate HTTP port (8000) with the Prometheus client library.

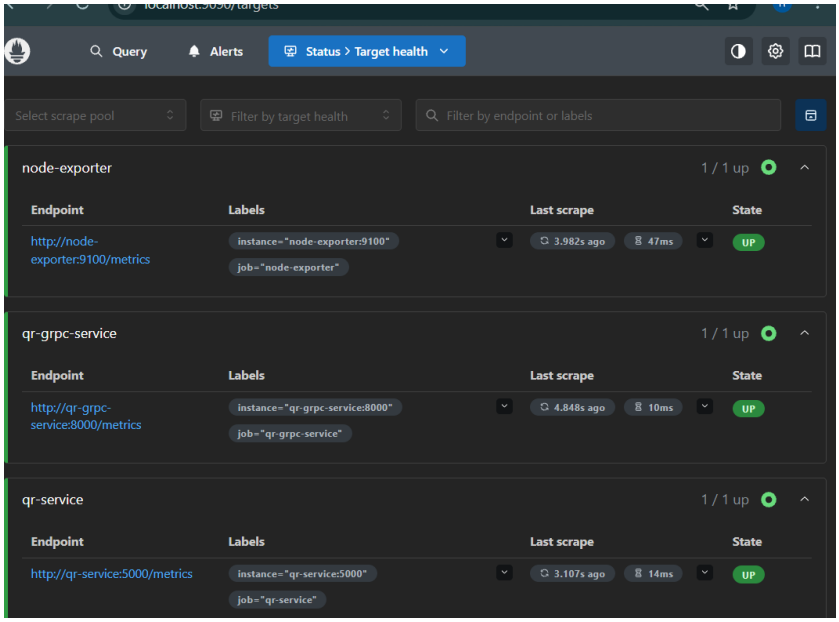


Figure C3. Prometheus Status > Target health: all three targets are UP (1/1 up).

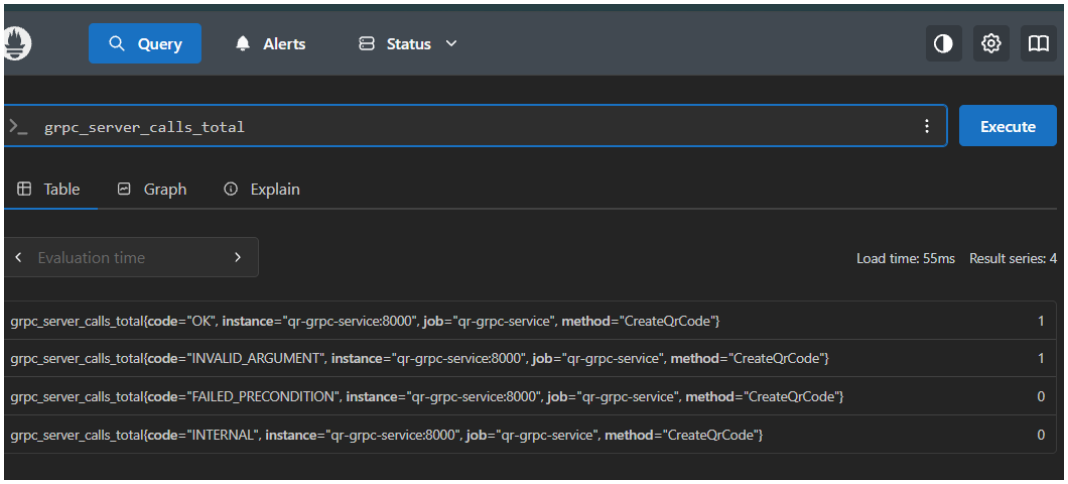


Figure C4. Prometheus query grpc_server_calls_total returns four series (OK, INVALID_ARGUMENT, FAILED_PRECONDITION, INTERNAL).

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker run --rm -d --name traffic -e QR_BASE_URL=http://host.docker.internal:5000
 -e GRPC_TARGET=host.docker.internal:50051 -e PYTHONPATH=/grpc -e PYTHONDONTWRITEBYTECODE=1 -v "${PWD}/clients:/clients" -v
"${PWD}/services/grpc-service:/grpc" -w /clients python:3.12-slim sh -c "pip install -q --root-user-action=ignore --disable-
pip-version-check -r requirements.txt -r /grpc/requirements.txt && python -u traffic.py 15"
c0f3c5d69b5171d6863fbd53a85d24ee48c82de482b687c2c7621f90073cccfb
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker ps --filter name=traffic
CONTAINER ID   IMAGE            COMMAND                  CREATED         STATUS         PORTS     NAMES
c0f3c5d69b51   python:3.12-slim   "sh -c 'pip install …"   2 minutes ago   Up 2 minutes             traffic
PS C:\Users\IBM\Desktop\ESD-Assignment-01> curl.exe -s http://localhost:8000/metrics | Select-String -Pattern "^grpc_server_
calls_total"

grpc_server_calls_total{code="OK",method="CreateQrCode"} 239.0
grpc_server_calls_total{code="INVALID_ARGUMENT",method="CreateQrCode"} 39.0
grpc_server_calls_total{code="FAILED_PRECONDITION",method="CreateQrCode"} 17.0
grpc_server_calls_total{code="INTERNAL",method="CreateQrCode"} 0.0
```

Figure C5. Traffic generator (clients/traffic.py) run in a container for 15 minutes. The gRPC counters afterwards: OK 239, INVALID_ARGUMENT 39, FAILED_PRECONDITION 17, INTERNAL 0.

C4. Grafana panels and PromQL

The dashboard `telemetry/grafana/dashboards/qr-service.json` is provisioned from the repo. New panels: both services and the A1 panels are in one dashboard. Rates use a **1 minute window** where I want to see quick changes under steady traffic; percentiles use **5 minutes** so that each window holds enough samples (with a 5 s scrape a 1m window has only ~12 points per series and gives jumpy percentiles).

Panel	PromQL	Why this panel
HTTP requests per second by route and status	sum by (route, status) (rate(http_requests_total{route!="/health"}[1m]))	Throughput and mix of 201/400/422/429
HTTP error rate % (1 min)	100 * sum(rate(http_requests_total{route!="/health",status=~"4..|5.."}[1m])) / sum(rate(http_requests_total{route!="/health"}[1m]))	Share of failed requests (health excluded)
gRPC calls per second by method and status	sum by (method, code) (rate(grpc_server_calls_total[1m]))	Same view for gRPC
gRPC error rate % (1 min)	100 * sum(rate(grpc_server_calls_total{code!="OK"}[1m])) / sum(rate(grpc_server_calls_total[1m]))	Share of non-OK calls
gRPC latency p50 and p95 (5 min)	histogram_quantile(0.5, sum by (le) (rate(grpc_server_call_duration_seconds_bucket[5m]))) and the same with 0.95	Typical vs slow-tail latency
gRPC call duration distribution (1 min)	sum by (le) (rate(grpc_server_call_duration_seconds_bucket[1m])) (format: heatmap)	Shows the whole latency shape over time, not only one percentile
gRPC calls in flight	grpc_server_in_flight	Concurrency
Idempotent replays per minute (HTTP vs gRPC)	sum by (job) (rate(qr_idempotent_replays_total[5m])) * 60	Shows retries being absorbed; one line per service
QR Codes Generated (Total), QR Requests In Progress	sum(qr_generated_total), qr_in_progress	Original A1 panels, recreated cleanly

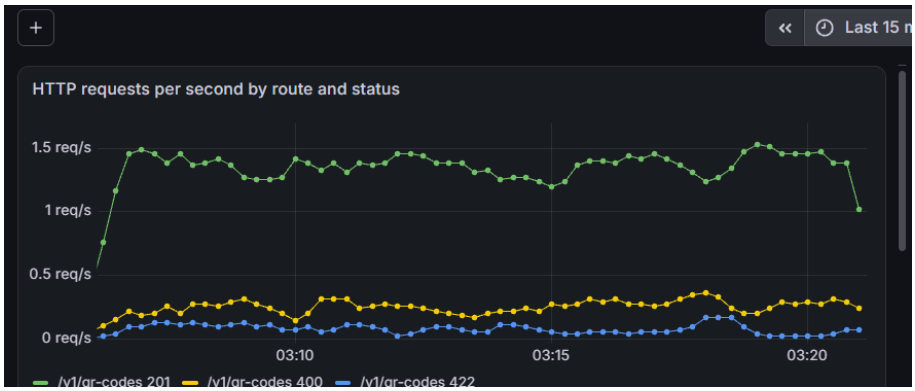


Figure C6. HTTP requests per second by route and status (201 dominant; 400 and 422 from the failure part of the traffic).

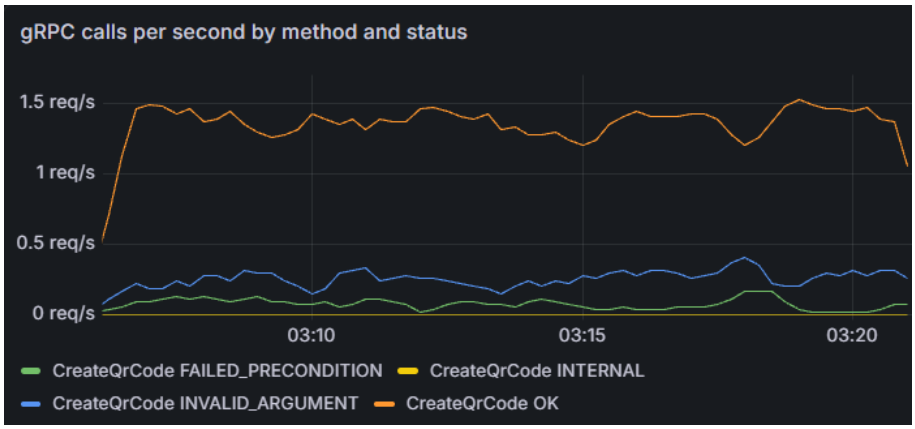


Figure C7. gRPC calls per second by method and status code (OK, INVALID_ARGUMENT, FAILED_PRECONDITION, INTERNAL).

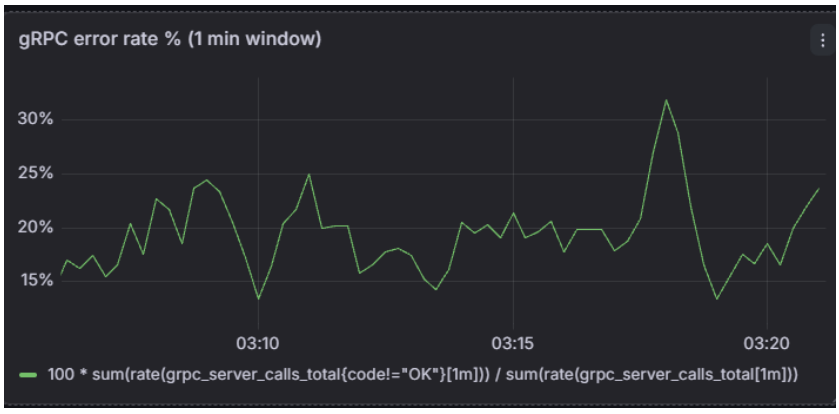


Figure C8. gRPC error rate % (1 min window): the failing share of the generated mix (the traffic script sends about a fifth bad requests on purpose).

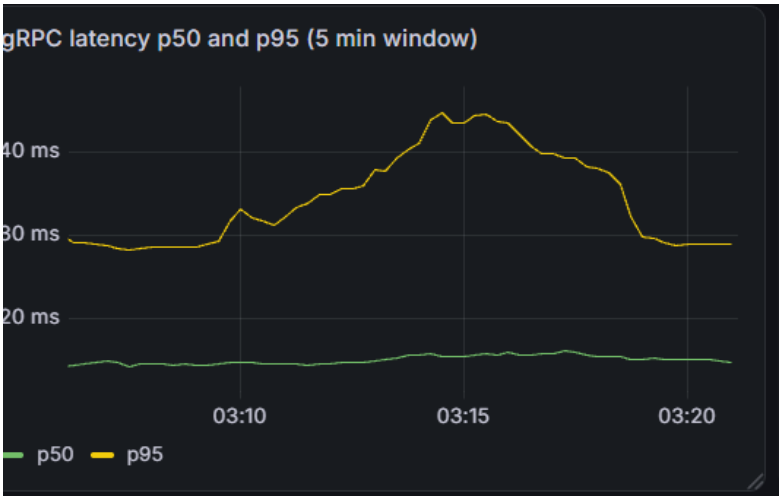


Figure C9. gRPC latency p50 and p95 (5 min window): p50 about 20 ms, p95 about 30-40 ms under load.

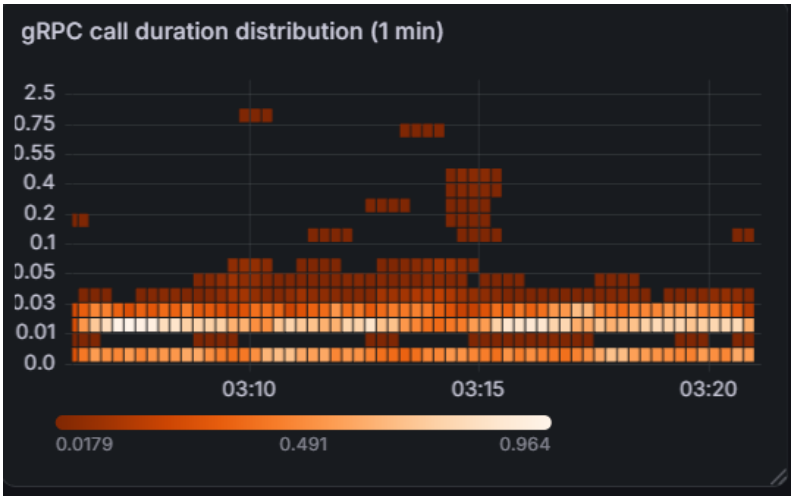


Figure C10. gRPC call duration distribution as a heatmap: most calls sit in the low-latency buckets, with occasional slower calls.



Figure C11. gRPC calls in flight: mostly 0, with a short blip to 1.

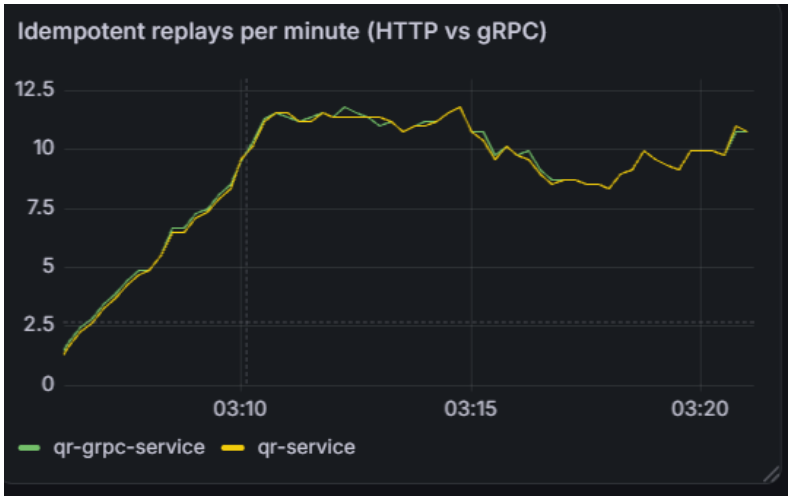


Figure C12. Idempotent replays per minute, qr-grpc-service vs qr-service: replays are counted on both sides.

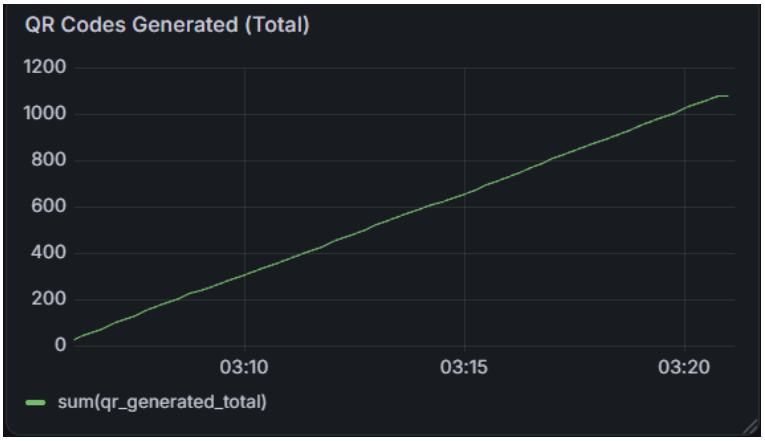


Figure C13. QR Codes Generated (Total) rises steadily during the traffic run.

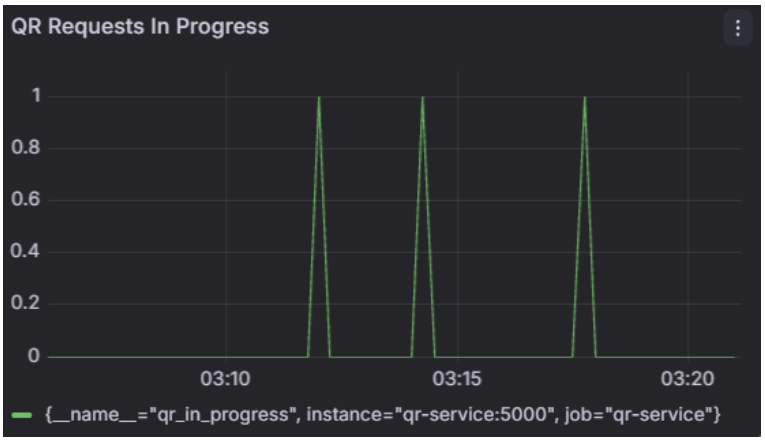


Figure C14. QR Requests In Progress: spikes only when a request is caught mid-handling.

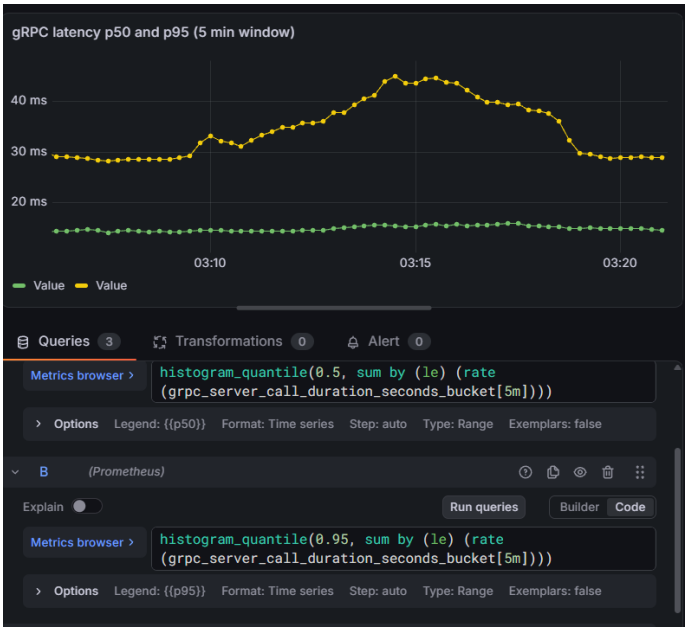


Figure C15. Panel editor for the latency panel: the two histogram_quantile queries (0.5 and 0.95 over grpc_server_call_duration_seconds_bucket[5m]) with legends p50 and p95.

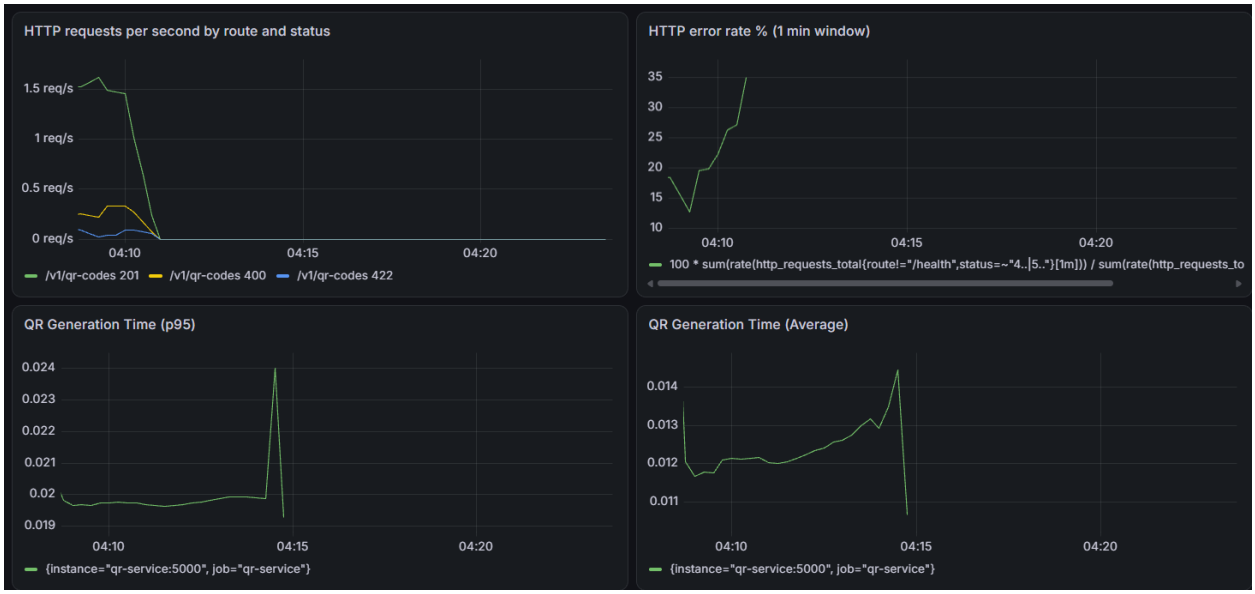


Figure C16. Whole dashboard (part 1): HTTP request rate, HTTP error rate, QR generation p95 and average. The right edge of the time range (about 04:11) shows traffic stopped, so lines fall to zero.



Figure C17. Whole dashboard (part 2): gRPC calls by status, gRPC error rate, latency p50/p95 and call-duration heatmap.

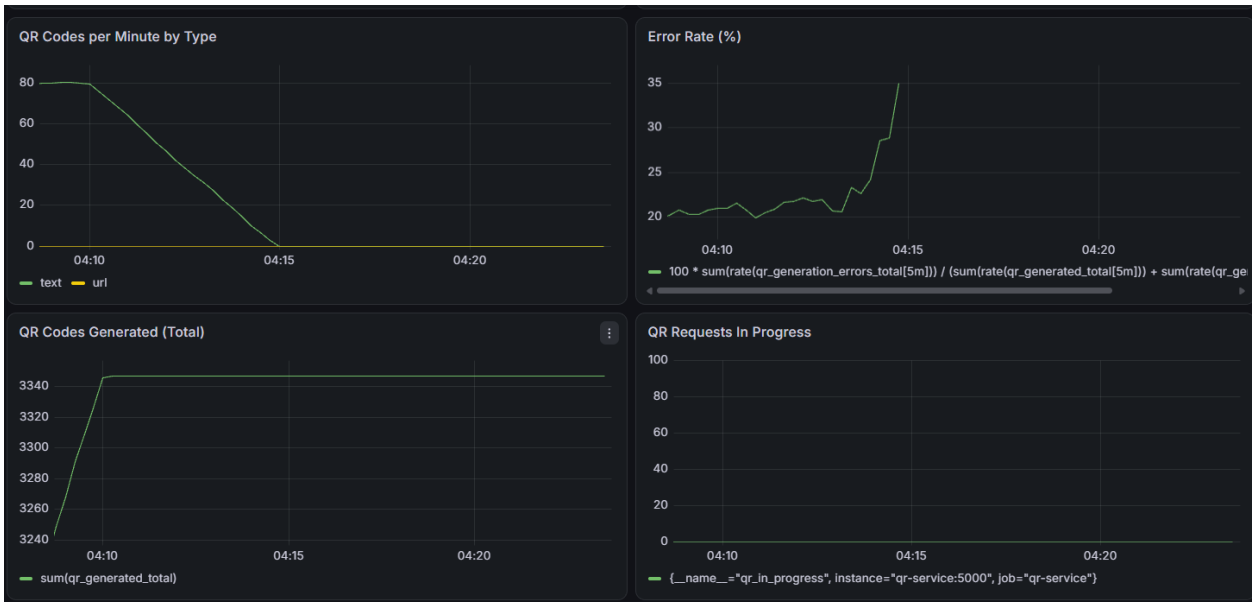


Figure C18. Whole dashboard (part 3): QR codes per minute by type, error rate, QR codes generated, requests in progress.

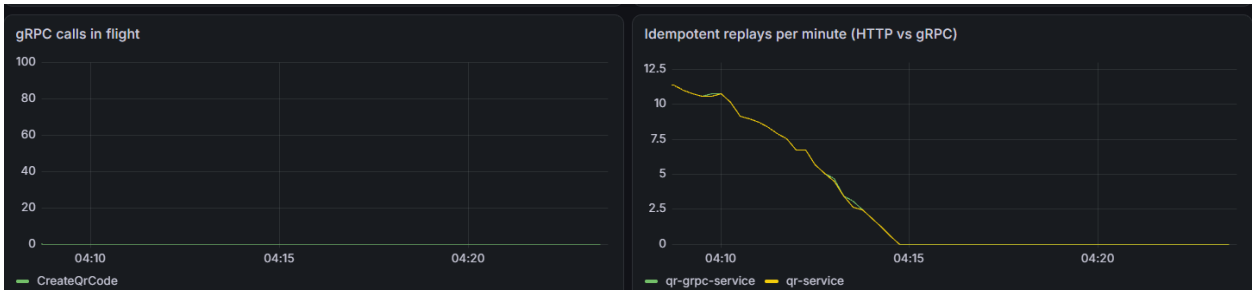


Figure C19. Whole dashboard (part 4): gRPC calls in flight and idempotent replays (HTTP vs gRPC).

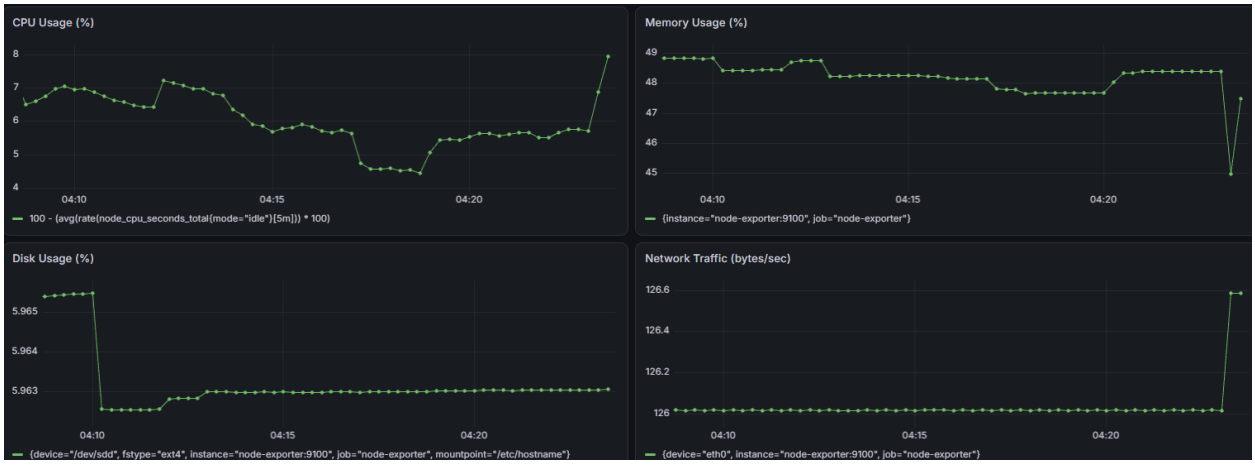


Figure C20. Whole dashboard (part 5): host panels from node-exporter (CPU, memory, disk, network).

C5. Client-side vs server-side duration

The two numbers answer different questions and are expected to differ:

- **Server-side** (`grpc_server_call_duration_seconds`, `qr_generation_duration_seconds`) is measured inside the handler. It covers validation, rendering and the database, but **not** the network, connection set-up, serialisation on the client, or time queued before the handler starts. It shows how fast the service works.
- **Client-side** (the load-test script and the deadline in the gRPC client) is measured by the caller from "send" to "reply". It includes everything above, so it is always at least as large as the server-side time. It shows what a user actually experiences. Example from my runs: the HTTP load test reported an average of 37 ms and p95 of 50 ms on the client, which includes the network hop from the host to the container.
- **How to read them together:** if server time is low but client time is high, the delay is outside the handler (network, queueing, overloaded caller). If both are high, the work itself is slow. The deadline (2 s for gRPC) and the 1 s timeout in the HTTP client are client-side limits, so they must be larger than the normal server-side p95.

In-flight gauge limitation: I chose a gauge because concurrency goes up and down. With a 5 s scrape and requests of 20-50 ms, a single sample catches an active request only rarely, so the panel mostly shows 0 and occasional 1 (Figures C11, C14). It is a coarse indicator; the heatmap and the p95 give a better picture of load.

Submission checklist and reuse

Required item	Where
Code	services/ (qr-service, grpc-service, common), clients/, docker-compose.yml
OpenAPI document	openapi/openapi.yaml
.proto and generation command	services/proto/qr.proto; command in Part B and README
Python client(s)	clients/qr_client.py, demo_reliability.py, grpc_client.py, compare_http_grpc.py, traffic.py
Tests	services/qr-service/tests/test_api.py (19), services/grpc-service/tests/test_grpc.py (6)
Prometheus config	telemetry/prometheus/prometheus.yml
Grafana dashboard	telemetry/grafana/dashboards/qr-service.json
README with run/test commands	README.md
Data volumes	Not committed (qr-data, Grafana, Elasticsearch data stay in Docker volumes)

Reused work: the gRPC workshop reference material (GRPC.md, provided in the course) guided the .proto structure, code generation and the stub/servicer layout; the HTTP API notes (APIs.md) and the assignment draft

guided the REST and OpenAPI parts. The Assignment 1 stack (Flask app, Prometheus, Grafana, ELK, node-exporter) is reused and extended. All other code, tests and configuration were written for this assignment.